

SKILL WEEK-2

Session - 3:

Task-1

To Apply Escape Sequences, Store Command History,
Navigate Previous Commands, Navigate Next Commands,
Update Input Buffer, Test Recall Functionality

CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX_BUFFER 1024
#define MAX_HISTORY 20

char history[MAX_HISTORY][MAX_BUFFER];
int history_count = 0;

/* Enable character-by-character input */
void enable_raw_mode(struct termios *old_terminal)
{
    struct termios new_terminal;

    tcgetattr(STDIN_FILENO, old_terminal);
    new_terminal = *old_terminal;

    new_terminal.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSANOW, &new_terminal);
}

/* Restore normal terminal mode */
void disable_raw_mode(struct termios *old_terminal)
{
    tcsetattr(STDIN_FILENO, TCSANOW, old_terminal);
}

/* Store command in history */
void store_command(const char *command)
{
    if (strlen(command) == 0)
        return;

    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        /* Shift old commands to make room */
        for (int i = 1; i < MAX_HISTORY; i++)
        {
```

```
            strcpy(history[i - 1], history[i]);
        }

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

/* Clear current line from terminal */
void clear_line(int length)
{
    printf("\r");

    for (int i = 0; i < length; i++)
        printf(" ");

    printf("\r");
}

/* Display history */
void show_history()
{
    printf("\n\nCommand History:\n");

    for (int i = 0; i < history_count; i++)
    {
        printf("%d. %s\n", i + 1, history[i]);
    }
}

/* Read command with arrow-key support */
void read_command(char *buffer)
{
    int position = 0;
    int history_position = history_count;

    buffer[0] = '\0';

    printf("os-shell> ");
    fflush(stdout);

    while (1)
    {
        char c;

        if (read(STDIN_FILENO, &c, 1) <= 0)
            continue;
```

```

/* ENTER key */
if (c == '\n' || c == '\r')
{
    buffer[position] = '\0';
    printf("\n");
    break;
}

/* BACKSPACE */
else if (c == 127 || c == 8)
{
    if (position > 0)
    {
        position--;
        buffer[position] = '\0';

        printf("\b \b");
        fflush(stdout);
    }
}

/* ESCAPE SEQUENCE */
else if (c == 27)
{
    char sequence[2];

    if (read(STDIN_FILENO, &sequence[0], 1) <= 0)
        continue;

    if (sequence[0] == '[')
    {
        if (read(STDIN_FILENO, &sequence[1], 1) <= 0)
            continue;

        /* UP ARROW: ESC [ A */
        if (sequence[1] == 'A')
        {
            if (history_count == 0)
                continue;

            if (history_position > 0)
                history_position--;

            clear_line(position);

            strcpy(buffer, history[history_position]);

```

```

        strcpy(buffer, history[history_position]);
        position = strlen(buffer);

        printf("%s", buffer);
        fflush(stdout);
    }

    /* DOWN ARROW: ESC [ B */
    else if (sequence[1] == 'B')
    {
        if (history_count == 0)
            continue;

        if (history_position < history_count - 1)
        {
            history_position++;

            clear_line(position);

            strcpy(buffer, history[history_position]);
            position = strlen(buffer);

            printf("%s", buffer);
        }
        else
        {
            history_position = history_count;

            clear_line(position);

            buffer[0] = '\0';
            position = 0;
        }

        fflush(stdout);
    }
}

/* NORMAL CHARACTER */
else
{
    if (position < MAX_BUFFER - 1)
    {
        buffer[position++] = c;
        buffer[position] = '\0';
    }
}

```

```

        putchar(c);
        fflush(stdout);
    }
    else
    {
        printf("\a");
        fflush(stdout);
    }
}
}

int main()
{
    struct termios old_terminal;
    char command[MAX_BUFFER];

    enable_raw_mode(&old_terminal);

    printf("=====\n");
    printf("      Simple OS Command Shell      \n");
    printf("=====\n");
    printf("Type commands and press Enter.\n");
    printf("Use UP/DOWN arrows for history.\n");
    printf("Type 'history' to view commands.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        read_command(command);

        if (strcmp(command, "exit") == 0)
        {
            break;
        }

        if (strcmp(command, "history") == 0)
        {
            show_history();
            continue;
        }

        store_command(command);

        printf("Command received: %s\n", command);
    }
}

```

```

    }

    disable_raw_mode(&old_terminal);

    printf("\nShell terminated.\n");

    return 0;
}

```

OUTPUT

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week2
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ nano task1.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ gcc -Wall -Wextra -o task1 task1.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ ./task1
```

```
=====
Simple OS Command Shell
=====
Type commands and press Enter.
Use UP/DOWN arrows for history.
Type 'history' to view commands.
Type 'exit' to quit.
```

```
os-shell> ls
Command received: ls
os-shell> pwd
Command received: pwd
os-shell> date
Command received: date
ls| hell>
```

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week2
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ nano task1.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ gcc -Wall -Wextra -o task1 task1.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ ./task1
```

```
=====
Simple OS Command Shell
=====
Type commands and press Enter.
Use UP/DOWN arrows for history.
Type 'history' to view commands.
Type 'exit' to quit.
```

```
os-shell> ls
Command received: ls
os-shell> pwd
Command received: pwd
os-shell> date
Command received: date
pwd| hell>
```

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week2
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ nano task1.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ gcc -Wall -Wextra -o task1 task1.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ ./task1
```

```
=====
Simple OS Command Shell
=====
Type commands and press Enter.
Use UP/DOWN arrows for history.
Type 'history' to view commands.
Type 'exit' to quit.
```

```
os-shell> ls
Command received: ls
os-shell> pwd
Command received: pwd
os-shell> date
Command received: date
datehell>
```

OBSERVATION :

The program successfully implements a simple command history mechanism using escape sequences. The entered commands are stored in a history buffer, and the Up (↑) and Down (↓) arrow keys are used to navigate through previously entered commands. The input buffer is updated with the recalled command, demonstrating proper handling of keyboard input and terminal escape sequences. The command history can also be displayed and recalled successfully.

Task - 2

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_CAPACITY 4
#define MAX_INPUT 100

/* =====
LINKED LIST NODE
===== */

typedef struct Node
{
    int value;
    struct Node *next;
} Node;

/* =====
DYNAMIC ARRAY
===== */

typedef struct
{
    int *data;
    size_t size;
    size_t capacity;
} DynamicArray;

/* Initialize dynamic array */
void initialize_array(DynamicArray *array)
{
    array->size = 0;
    array->capacity = INITIAL_CAPACITY;

    array->data = malloc(
        array->capacity * sizeof(int)
    );

    if (array->data == NULL)
    {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
}
```

```

/* Resize dynamic array */
void resize_array(DynamicArray *array)
{
    array->capacity *= 2;

    int *temp = realloc(
        array->data,
        array->capacity * sizeof(int)
    );

    if (temp == NULL)
    {
        printf("Memory reallocation failed.\n");

        free(array->data);
        exit(EXIT_FAILURE);
    }

    array->data = temp;

    printf("Array resized. New capacity = %zu\n",
        array->capacity);
}

/* Add element safely */
void add_element(DynamicArray *array, int value)
{
    /* Prevent array overflow */
    if (array->size >= array->capacity)
    {
        resize_array(array);
    }

    array->data[array->size] = value;
    array->size++;
}

/* Display dynamic array */
void display_array(const DynamicArray *array)
{
    printf("\nDynamic Array:\n");

    for (size_t i = 0; i < array->size; i++)
    {
        printf("%d ", array->data[i]);
    }
}

```

```

        printf("%d ", array->data[i]);
    }

    printf("\n");
}

/* Free dynamic array */
void free_array(DynamicArray *array)
{
    free(array->data);

    array->data = NULL;
    array->size = 0;
    array->capacity = 0;
}

/* =====
LINKED LIST FUNCTIONS
===== */

/* Insert node at end */
void insert_node(Node **head, int value)
{
    Node *new_node = malloc(sizeof(Node));

    if (new_node == NULL)
    {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }

    new_node->value = value;
    new_node->next = NULL;

    if (*head == NULL)
    {
        *head = new_node;
        return;
    }

    Node *current = *head;

    while (current->next != NULL)
    {
        current = current->next;
    }

```

GNU nano 8.7.1

```
    current->next = new_node;
}

/* Display linked list */
void display_list(Node *head)
{
    printf("\nLinked List:\n");

    Node *current = head;

    while (current != NULL)
    {
        printf("%d -> ", current->value);

        current = current->next;
    }

    printf("NULL\n");
}

/* Free linked list */
void free_list(Node **head)
{
    Node *current = *head;

    while (current != NULL)
    {
        Node *next = current->next;

        free(current);

        current = next;
    }

    *head = NULL;
}

/* =====
SAFE STRING BUFFER
===== */
```

```
/* -----  
Safe Input  
----- */  
  
safe_input();  
  
/* -----  
Linked List  
----- */  
  
printf("\nCreating linked list...\n");  
  
insert_node(&head, 10);  
insert_node(&head, 20);  
insert_node(&head, 30);  
insert_node(&head, 40);  
  
display_list(head);  
  
/* -----  
Free memory  
----- */  
  
printf("\nReleasing allocated memory...\n");  
  
free_array(&array);  
  
free_list(&head);  
  
printf("Memory released successfully.\n");  
  
return 0;  
}
```

```

void safe_input()
{
    char buffer[MAX_INPUT];

    printf("\nEnter a string (maximum %d characters): ",
        MAX_INPUT - 1);

    if (fgets(buffer, sizeof(buffer), stdin) != NULL)
    {
        buffer[strcspn(buffer, "\n")] = '\0';

        printf("You entered: %s\n", buffer);
    }
}

/* =====
   MAIN
   ===== */

int main()
{
    DynamicArray array;

    Node *head = NULL;

    /* -----
       Dynamic Array
       ----- */

    initialize_array(&array);

    printf("Adding elements to dynamic array...\n");

    for (int i = 1; i <= 10; i++)
    {
        add_element(&array, i * 10);
    }

    display_array(&array);
}

```

OUTPUT

```
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ nano task2.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ gcc -Wall -Wextra -g -o task2 task2.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ ./task2
Adding elements to dynamic array...
Array resized. New capacity = 8
Array resized. New capacity = 16

Dynamic Array:
10 20 30 40 50 60 70 80 90 100

Enter a string (maximum 99 characters): hello
You entered: hello

Creating linked list...

Linked List:
10 -> 20 -> 30 -> 40 -> NULL

Releasing allocated memory...
Memory released successfully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ |
```

OBSERVATION :

The program successfully demonstrates dynamic memory allocation and management in C. Dynamic arrays are allocated using `malloc()` and resized using `realloc()` when the capacity is reached, preventing buffer overflow. A linked list is created using dynamically allocated nodes and properly traversed. The allocated memory is released using `free()`. Valgrind verification confirms that the program executes without memory leaks or invalid memory access.

Session - 4 :

Task -1

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output

CODE :

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

typedef struct {
    char value[20];
    char type[20];
} Token;

int main() {
    char input[MAX];
    Token tokens[50];
    int n = 0, i = 0;

    printf("Enter command: ");
    fgets(input, MAX, stdin);

    while (input[i] != '\0') {
        if (isspace(input[i])) {
            i++;
            continue;
        }

        if (input[i] == '|') {
            strcpy(tokens[n].value, "|");
            strcpy(tokens[n].type, "PIPE");
            n++;
            i++;
        }
        else if (input[i] == '<' || input[i] == '>') {
            tokens[n].value[0] = input[i];
            tokens[n].value[1] = '\0';
            strcpy(tokens[n].type, "REDIRECT");
            n++;
            i++;
        }
        else {
            int j = 0;

            while (input[i] != '\0' &&
                !isspace(input[i]) &&
                input[i] != '|' &&
                input[i] != '<' &&
                input[i] != '>') {
                Read 66 lines
            }

```

```
                tokens[n].value[j++] = input[i++];
            }

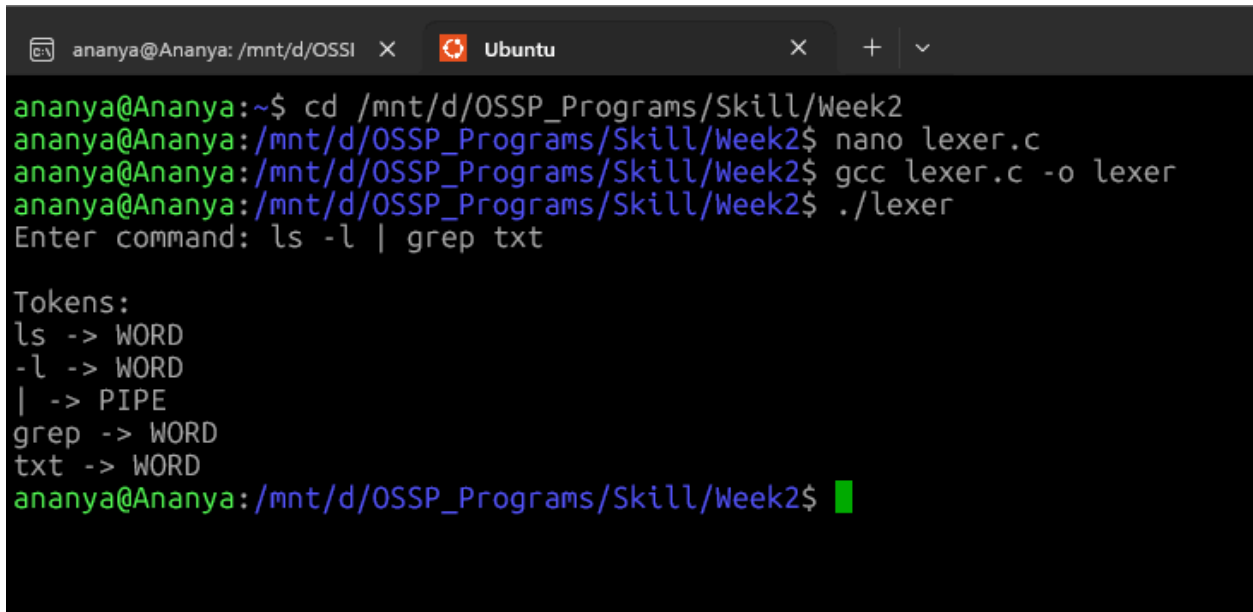
            tokens[n].value[j] = '\0';
            strcpy(tokens[n].type, "WORD");
            n++;
        }
    }

    printf("\nTokens:\n");

    for (i = 0; i < n; i++)
        printf("%s -> %s\n",
            tokens[i].value,
            tokens[i].type);

    return 0;
}
```

OUTPUT:

A terminal window titled 'ananya@Ananya: /mnt/d/OSSI' with a Ubuntu logo. The user enters a series of commands: 'cd /mnt/d/OSSP_Programs/Skill/Week2', 'nano lexer.c', 'gcc lexer.c -o lexer', and './lexer'. The program outputs 'Enter command: ls -l | grep txt'. It then displays a list of tokens: 'Tokens: ls -> WORD', '-l -> WORD', '| -> PIPE', 'grep -> WORD', and 'txt -> WORD'. The prompt returns to 'ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week2\$' with a green cursor.

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week2
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ nano lexer.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ gcc lexer.c -o lexer
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ ./lexer
Enter command: ls -l | grep txt

Tokens:
ls -> WORD
-l -> WORD
| -> PIPE
grep -> WORD
txt -> WORD
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$
```

OBSERVATION :

The program successfully performs basic lexical analysis by reading the given input and dividing it into individual tokens. It identifies different types of tokens such as words, pipes, and redirection operators while ignoring unnecessary whitespace. Each token is stored with its corresponding type and displayed as the token stream. Thus, the program demonstrates tokenization, delimiter identification, whitespace handling, and generation of parsing output.

Task - 2

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

CODE :

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];
    char *token;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Error: Empty command\n");
        return 0;
    }

    printf("\nParse Tree:\n");

    token = strtok(input, " ");

    if (token == NULL) {
        printf("Invalid command\n");
        return 0;
    }

    printf("COMMAND\n");
    printf("  |\n");
    printf("  +-- %s\n", token);

    while ((token = strtok(NULL, " ")) != NULL) {
        if (strcmp(token, "|") == 0) {
            printf("  |\n");
            printf("  +-- PIPE\n");
        }
        else {
            printf("      +-- %s\n", token);
        }
    }

    printf("\nSyntax is valid.\n");

    return 0;
}
```

OUTPUT :

```
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ nano parser.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ gcc parser.c -o parser
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ ./parser
Enter command: ls -l | grep txt

Parse Tree:
COMMAND
|
+-- ls
    +-- -l
    |
    +-- PIPE
        +-- grep
        +-- txt

Syntax is valid.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week2$ █
```

OBSERVATION :

The program successfully implements basic parser logic by processing the input command and identifying its different components. It generates a simple parse tree containing the command, arguments, and pipe operators. The program also checks for an empty command and reports an error when no input is provided. Thus, the program demonstrates basic syntax validation, parse tree generation, handling of commands, and preparation of the input for further execution.